

# Python 코드 핵심 정리

day01 ~ day07 | 알고리즘분석 실습

## day02-01 — 자료형

### [is vs ==]

```
m1 = 1000
m2 = 1000
print(m1 is m2)
print(m1 == m2)
```

출력  
False  
True

※ *is*는 메모리 주소 비교, *==*는 값 비교

### [문자열 인터닝]

```
a = "String"
b = "String"
print(a is b)
b += "1"
b = b[:-1]
print(a is b)
```

출력  
True  
False

※ 수정 전엔 같은 메모리 공유, 수정 후엔 새 객체 생성

### [단락 평가 (short-circuit)]

```
print(0 or 0)
print(0 or 1)
print(0 or 2)
print(1 and 0)
print(1 and 1)
print(1 and 2)
```

출력  
0  
1  
2  
0  
1  
2

※ *or*: 왼쪽 **False**이면 오른쪽 반환 / *and*: 왼쪽 **True**이면 오른쪽 반환

## day02-02 — 리스트 / 딕셔너리 / 집합

---

### [리스트 슬라이싱]

```
c = [1,2,3,4,5,6,7,8,9]
print(c[0:-1:2])
```

출력

```
[1, 3, 5, 7]
```

※[시작:끝:간격] — 간격을 주면 건너뛰며 추출

### [insert / append / extend]

```
a = [0,1,2,3,4,5,6,7,8,9]
a.insert(0, -1)
a.insert(-1, 10)
print(a)
```

```
a = [0,1,2,3,4,5,6,7,8,9]
a.append(10)
print(a)
```

```
a = [0,1,2,3,4,5,6,7,8,9]
b = [10,11]
a.extend(b)
print(a)
```

출력

```
[-1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 9]
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]
```

※*append*는 요소 하나, *extend*는 리스트 전체 이어붙임

### [딕셔너리 get()]

```
a = {"one":"하나", "two":"둘", "three":"셋"}
print(a["one"])
print(a.get("two"))
print(a.get("four", "넷"))
```

출력

```
하나
```

```
둘
```

```
넷
```

※*get()*은 키 없으면 기본값 반환. *a["four"]*는 *KeyError* 발생

### [집합 연산]

```
a = {1,2,3}
b = {1,3,5,7,9}
print(a | b)
print(a & b)
print(a - b)
```

출력

```
{1, 2, 3, 5, 7, 9}
```

```
{1, 3}
```

```
{2}
```

※| 합집합 / & 교집합 / - 차집합

## day02-03 — 함수

---

### [기본값 / 가변 인자]

```
def add(a, b=10):  
    return a + b  
print(add(1))  
print(add(1, 2))
```

```
def add(*num):  
    return sum(num)  
print(add(1, 2, 3))
```

출력

```
11  
3  
6
```

※ `b=10` 기본값 설정 / `*num`은 인자 몇 개든 튜플로 받음

### [함수를 변수에 담기]

```
def add(a, b):  
    return a + b  
plus = add  
print(plus(3, 4))
```

```
def func(x, y, z):  
    return x(y, z)  
print(func(plus, 3, 4))
```

출력

```
7  
7
```

※ 함수도 변수처럼 전달 가능 (1급 객체)

### [lambda + key]

```
L = [[9,1],[8,2],[7,3],[6,4]]  
print(max(L, key=lambda x: x[0]))  
print(max(L, key=lambda x: x[1]))  
print(min(L, key=lambda x: x[0]))
```

출력

```
[9, 1]  
[6, 4]  
[6, 4]
```

※ `lambda`를 `key`로 쓰면 비교 기준을 자유롭게 설정

## day03-01 — numpy / pandas

### [ndarray 기본 생성 및 dtype 확인]

```
import numpy as np
a = np.array([1, 2, 3, 4])
b = np.array([1, 2, 3.1, 4])
c = np.array([1, '2', 3, 4])
print(a.dtype)
print(b.dtype)
print(c.dtype)
```

```
출력
int32
float64
<U21 (문자열로 통일됨)
```

※ 하나라도 실수면 **float**, 문자열이 섞이면 전부 문자열로 변환됨 (dtype 자동 결정)

### [ndim / shape]

```
a1 = np.array([1, 2, 3, 4, 5])
a2 = np.array([[1,2,3,4],[1,2,3,4]])
a3 = np.array([[[1,2,3],[1,2,3]],[[1,2,3],[1,2,3]],[[1,2,3],[1,2,3]]])
print(a1.ndim, a1.shape)
print(a2.ndim, a2.shape)
print(a3.ndim, a3.shape)
```

```
출력
1 (5,)
2 (2, 4)
3 (3, 2, 3)
```

※ **ndim**은 차원 수, **shape**은 각 축의 크기. (행, 열) 순서

### [zeros / ones / arange]

```
print(np.zeros(10))
print(np.ones(10))
print(np.arange(1, 10, 2))
```

```
출력
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
[1 3 5 7 9]
```

※ **zeros/ones**로 초기 배열 생성, **arange**는 **range**처럼 배열 생성

### [Series 생성]

```
import pandas as pd
e = pd.Series([1, 2, 3, 4, 5])
f = pd.Series([1200, 3500, 2100, 9600],
              index=['사과', '딸기', '키위', '메론'])
print(e[0])
print(f['딸기'])
```

```
출력
1
3500
```

※ **Series**는 인덱스+값 쌍. 숫자 인덱스 또는 문자열 인덱스 모두 가능

### [DataFrame 생성 및 기본 메서드]

```
df = pd.DataFrame({
    '이름': ['피카츄', '라이츄', '리자드', '리자몽'],
    '속성': ['전기', '전기', '불', '불'],
    '순위': [5, 2, 6, 1]
})
df.info()
df.shape
print(len(df))
print(len(df.columns))
```

```

출력
<class 'pandas.core.frame.DataFrame'>
(4, 3)
4   # 행 수
3   # 열 수

```

※ `info()`로 전체 구조 확인, `shape`으로 (행, 열) 크기 확인

### [열 선택]

```

df['속성']           # Series로 반환
df[['속성']]         # DataFrame으로 반환
df[['이름', '순위']] # 여러 열 선택
df['이름'].values     # numpy 배열로 반환

```

```

출력
전기 / 전기 / 불 / 불   (Series)
열 하나짜리 DataFrame
이름, 순위 두 열짜리 DataFrame
['피카츄' '라이츄' '리자드' '리자몽']

```

※ [] 하나면 **Series**, [[]] 두 겹이면 **DataFrame**. 자주 혼동하는 포인트

## day04 — 반복 / 입력 / numpy

---

### [map + split 입력]

```
in_value = list(map(int, input().split()))
# 입력: 7 1 4 3 6
print(in_value)
print(type(in_value))

출력
[7, 1, 4, 3, 6]
<class 'list'>
```

※ 공백으로 구분된 숫자 입력을 정수 리스트로 변환

### [map 객체 주의]

```
in_value = map(int, input().split())
# map 객체는 한 번 순회하면 소진됨
# min(in_value) 후 max(in_value) → 오류!

출력
ValueError: min() iterable argument is empty
```

※ `map()`은 이터레이터. 두 번 쓰려면 `list()` 변환 필수

### [1~N 합 (3가지 방법)]

```
# n = 10
result = 0
for i in range(1, 11):
    result += i
print(result)

print(sum(range(1, 11)))

print((1 + 10) * 10 / 2)

출력
55
55
55.0
```

※ 세 가지 모두 같은 결과. 공식이 가장 빠름  $O(1)$

### [numpy 배열 연산]

```
import numpy as np
a = np.array(range(1, 11))
b = np.array(range(10, 0, -1))
print(a)
print(b)
print(a + b)

출력
[ 1  2  3  4  5  6  7  8  9 10]
[10  9  8  7  6  5  4  3  2  1]
[11 11 11 11 11 11 11 11 11 11]
```

※ `numpy` 배열은 요소끼리 바로 연산 가능

## day05-01 — 재귀함수

### [무한 재귀 (기저조건 없음)]

```
def recurse():
    print('*')
    recurse()

recurse()

출력
* * * * * ... (RecursionError: maximum recursion depth exceeded)
```

※ 기저조건 없으면 무한 호출 → 스택 오버플로우 발생

### [기저조건 있는 재귀]

```
def recurse(count):
    if count == 0:
        print('.') # 기저조건 (탈출 조건)
    else:
        print('*')
        recurse(count-1) # 자기 자신을 count-1로 호출

recurse(4)

출력
*
*
*
*
.
```

※ 기저조건(`count == 0`)이 있어야 재귀가 멈춤. 재귀의 핵심

### [재귀로 팩토리얼 구현]

```
def factorial(n):
    print(n)
    if n == 1:
        return 1
    else:
        return n * factorial(n-1)

print(factorial(4))

출력
4
3
2
1
24
```

※  $4! = 4 \times 3 \times 2 \times 1 = 24$ . 재귀 호출 순서를 출력으로 확인 가능

### [반복문으로 팩토리얼 구현]

```
factorial = 1
for i in range(1, 5):
    print(i)
    factorial *= i
print(factorial)

출력
1
2
3
4
24
```

※ 재귀와 반복문은 같은 결과. 반복문이 메모리 효율적

## day05-02 — 배열로 연결리스트 흉내

---

### [중간 삽입 (뒤에서부터 밀기)]

```
katok = ['철수', '영희', '민수']
katok.append(None)      # 자리 확보
katok[3] = katok[2]     # 뒤로 복사
katok[2] = None         # 빈 자리 만들기
katok[2] = '영수'       # 새 값 삽입
print(katok)

출력
['철수', '영수', '영희', '민수']
```

※ 배열에서 중간 삽입 시 뒤 요소를 먼저 밀어야 함

### [중간 삭제 (앞으로 당기기)]

```
katok[2] = katok[3]     # 앞으로 당기기
del(katok[3])           # 마지막 빈 자리 제거
print(katok)

출력
['철수', '영수', '민수']
```

※ 삭제 시 뒤 요소를 앞으로 당긴 후 마지막 요소 제거



## day06-01 — 연결리스트

---

### [Node 클래스]

```
class Node():
    def __init__(self):
        self.data = None
        self.link = None    # 다음 노드 가리킴
```

※ *data*는 값, *link*는 다음 노드의 주소. 연결리스트의 기본 단위

### [노드 순회]

```
current = head
while current.link != None:
    current = current.link
    print(current.data, end=' ')
```

출력

다현 정연 썬위 사나 지효

※ *link*를 따라가며 순회. *link*가 *None*이면 마지막 노드

### [insertNode / deleteNode]

```
insertNode('다현', '화사')
printNodes(head)
```

```
deleteNode('다현')
printNodes(head)
```

출력

화사 다현 정연 썬위 사나 지효

화사 정연 썬위 사나 지효

※ 삽입은 포인터 연결, 삭제는 포인터 건너뛰

## day06-02 — 스택 (Stack)

---

### [push / pop]

```
def push(data):
    global SIZE, stack, top
    if top >= SIZE-1:
        print('Stack Full'); return
    top += 1
    stack[top] = data

def pop():
    global SIZE, stack, top
    if top == -1:
        print('Stack Empty'); return None
    data = stack[top]
    stack[top] = None
    top -= 1
    return data
```

출력

['다현', '정연', '썬위', '사나', '지효'] 4

사나

['다현', '정연', '썬위', None, None]

※ *top=-1*이면 빈 스택, *SIZE-1*이면 가득 찬 스택

## day07 — 괄호 균형 검사 (스택 응용)

---

### [괄호 검사 로직]

```
bracket_stack = []
answer = True
for j in bracket:
    if j in '([':
        bracket_stack.append(j)
    elif j == ')':
        if bracket_stack[-1] == '(':
            bracket_stack.pop()
        else:
            answer = False
    elif j == ']':
        if bracket_stack[-1] == '[':
            bracket_stack.pop()
        else:
            answer = False
if answer and not bracket_stack:
    print('yes')
else:
    print('no')
```

#### 출력

```
So when I die (the [first]...) is yes
([ (([ ( [ ] ) ( ) (( )) ] ) ) ]. is yes
```

※ 열린 괄호 *push*, 닫힌 괄호는 짝 확인 후 *pop*. 마지막에 스택이 비어야 *yes*